



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W2D1&W2D2: Linux下的网络 编程基础

Linux下Web Server的设计

协议

- 要想让两台计算机进行通信，必须使它们采用相同的信息交换规则
- 计算机网络中用于规定信息的格式以及如何发送和接收信息的一套规则称为网络协议（network protocol）或通信协议（communication protocol），简称协议（Protocol）

其实就是用来说明网络中传输的二进制数据具体含义和作用的上下文



协议

- 网络协议三要素：
 - 语法：即数据与控制信息的结构或格式
 - 语义：即需要发出何种控制信息，完成何种动作以及做出何种响应
 - 时序：即事件实现顺序的详细说明
- PDU：Protocol Data Unit 协议数据单元，即协议对应的具体二进制数据



分层模型

- 为了减少网络协议设计的复杂性，网络设计者并不是设计一个单一、巨大的协议来为所有形式的通信规定完整的细节
- 而是采用**把通信问题划分为许多个小问题**，然后为每个小问题设计一个单独的协议的方法，这样做使得每个协议的设计、分析、编码和测试都比较容易
- 分层模型（layering model）是一种用于开发网络协议的设计方法。本质上，分层模型描述了把通信问题分为几个小问题（称为层次）的方法，每个小问题对应于一层。



分层模型

- 网络中同等层之间的通信规则就是该层使用的协议，如有关第N层的通信规则的集合，就是第N层的协议。
- 而同一计算机的不同功能层之间的通信规则称为接口（interface），在第N层和第（N+ 1）层之间的接口称为N /（N+ 1）层接口。
- 协议是不同机器同等层之间的通信约定，而接口是同一机器相邻层之间的通信约定。



分层优势

- 各层之间是独立的。下层对于上层而言是个黑匣子，上层不需要知道下层是如何实现的，只需要知道下层提供什么功能怎么使用功能。
- 灵活性好。当任何一层发生变化时，只要层间接口保持不变，则其它层不受影响。
- 结构上可分割开。各层都可以采用最合适的技术来实现。
- 易于实现和维护。使得实现和调试一个庞大而又复杂的系统变得易于处理。

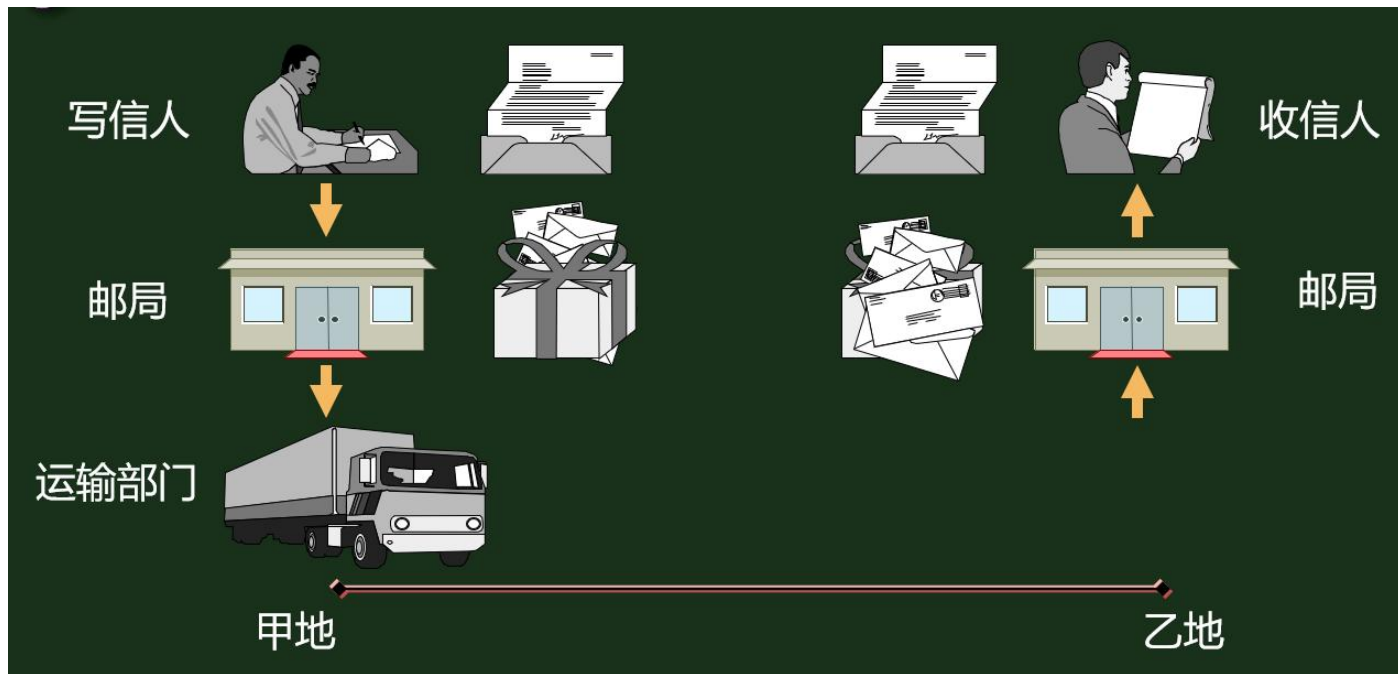


分层优势

- 促进标准化工作。因为每一层的功能及其所提供的服务都已有了精确的说明。
- 通常各层需要完成的功能包括以下一种或几种任务：
 - ① 差错控制：使得通信更加可靠
 - ② 流量控制：发送速率和接收速率基本同步
 - ③ 分段和重装：将数据分组发送，接收端再还原
 - ④ 复用和分用：复用底层连接，在接收端再进行分用
 - ⑤ 连接建立和释放：逻辑连接的建立和释放



现实实例-邮局体系



OSI/RM

- 网络的快速发展和广泛应用迫切需要采用不同体系结构的网络能够互连从而交换信息。
- ISO(国际标准化组织), 1977年试图制定一个统一的体系结构, 即OSI/RM:Open Systems Interconnection Reference Mode 开放系统互连参考模型
- 1983年OSI定稿, 同年TCP/IP协议作为因特网上所有主机间的共同协议, 从此以后被作为一种必须遵守的规则被肯定和应用, 导致OSI未能商业化。



OSI/RM



OSI 七层模型图解



TCP/IP

- **TCP: Transmission Control Protocol, 传输控制协议**
- **IP: Internet Protocol, 因特网互联协议**
- 这两个协议是因特网最核心的协议
- 因特网中传输层、网际层所有协议的统称, 也叫TCP/IP协议族或TCP/IP协议栈, 简称TCP/IP

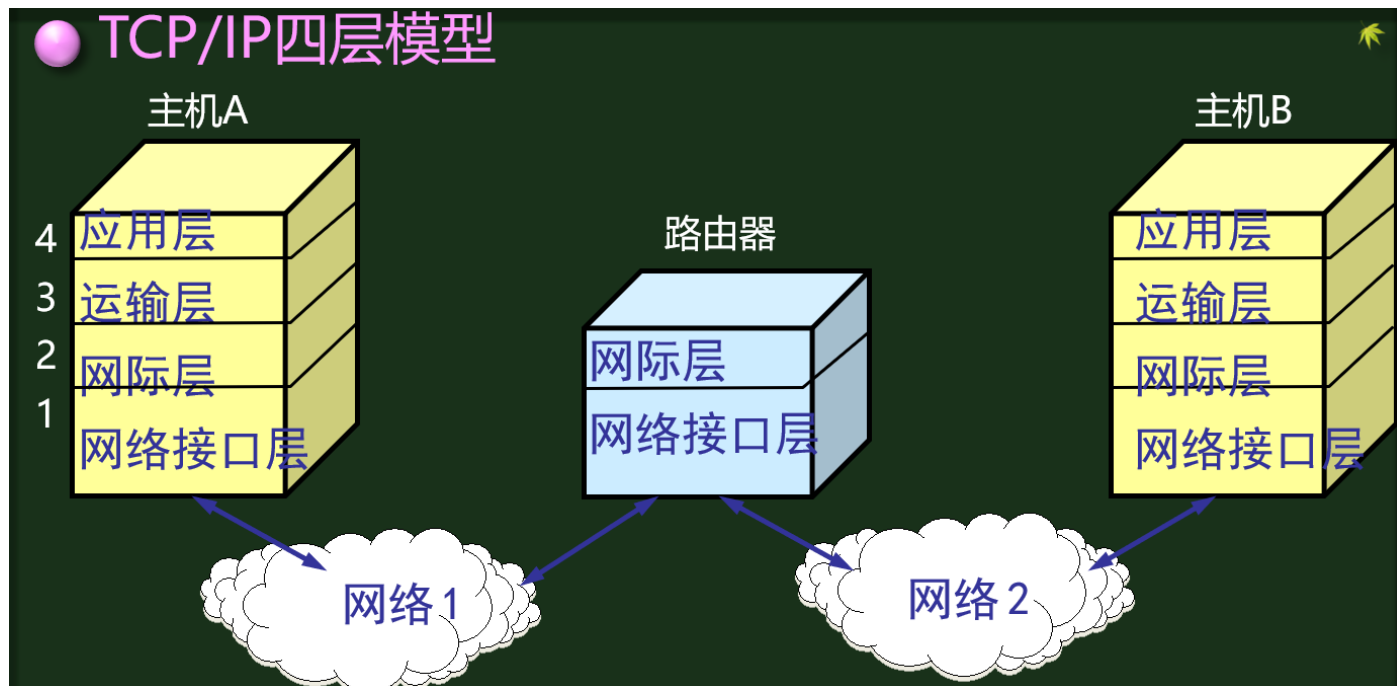


TCP/IP

- TCP/IP只有四层结构：
 - 应用层：直接为用户的应用进程提供服务
 - 运输层：负责两个主机中进程之间的通信
 - 网际层：负责不同主机之间的通信
 - 网络接口层：没有具体内容
- 计算机科学中为了研究方便一般采用5层协议模型研究TCP/IP



TCP/IP

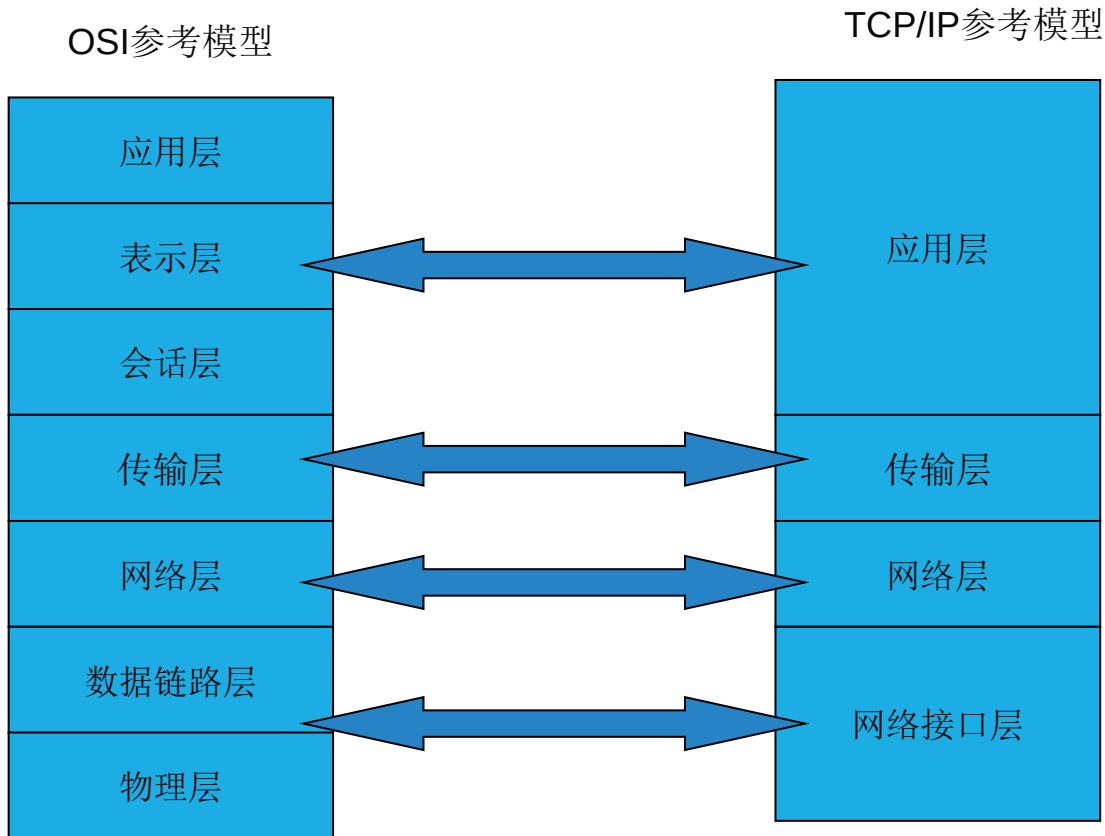


● TCP/IP五层模型

- ◆ 应用层：直接为用户的应用进程提供服务
- ◆ 传输层：负责两个主机中进程之间的通信
- ◆ 网络层：负责不同主机之间的通信
- ◆ 数据链路层：负责相邻设备间的通信
- ◆ 物理层：负责传输比特流



TCP/IP协议栈



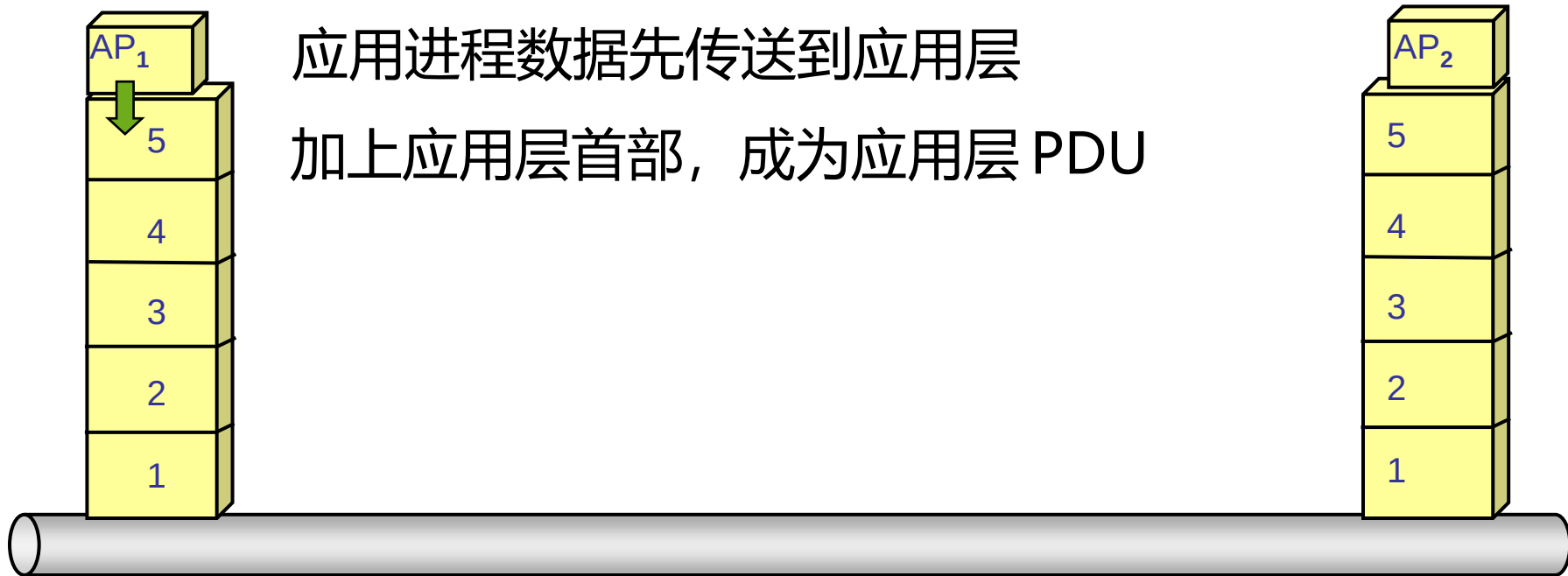
OSI参考模型与TCP/IP参考模型对应关系



TCP/IP通信过程

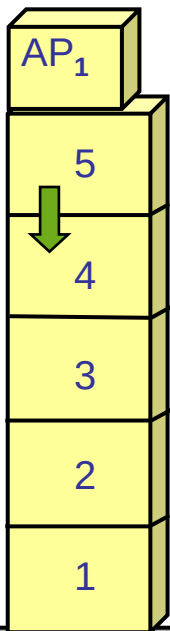
主机 1

主机 2



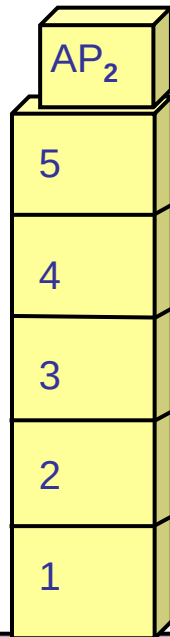
TCP/IP通信过程

主机 1



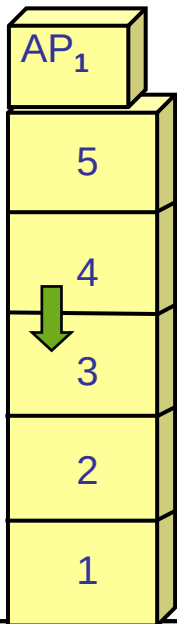
应用层 PDU 再传送到运输层
加上运输层首部，成为运输层报文

主机 2



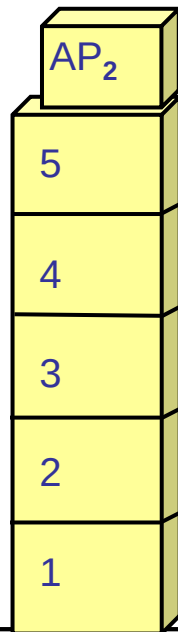
TCP/IP通信过程

主机 1



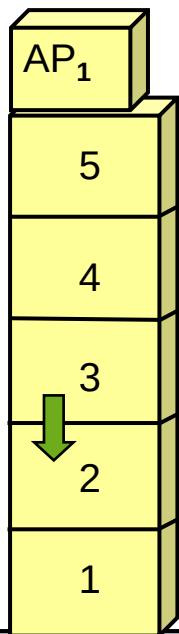
运输层报文再传送到网络层
加上网络层首部,
成为 IP 数据报 (或分组)

主机 2



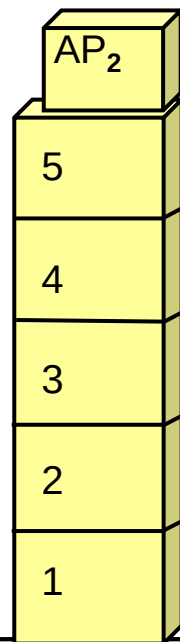
TCP/IP通信过程

主机 1



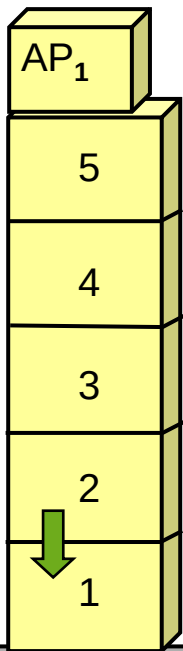
IP 数据报再传送到数据链路层
加上链路层首部和尾部，
成为数据链路层帧

主机 2

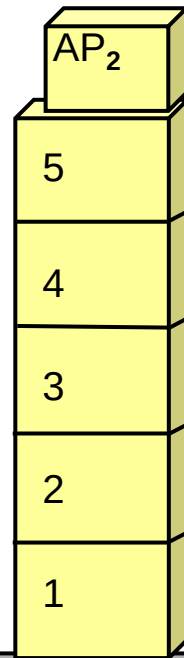


TCP/IP通信过程

主机 1



主机 2



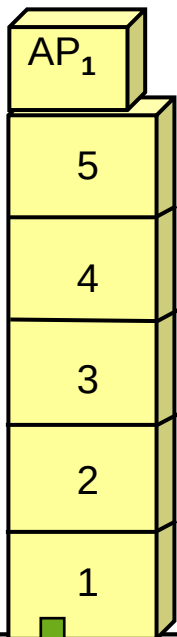
数据链路层帧再传送到物理层

最下面的物理层把比特流传送到物理媒体

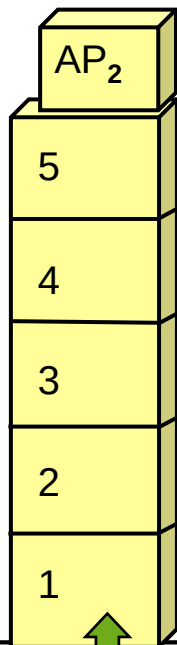


TCP/IP通信过程

主机 1



主机 2

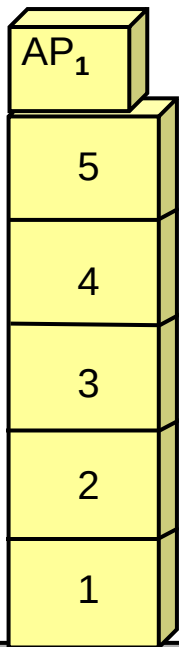


电信号（或光信号）在物理媒体中传播
从发送端物理层传送到接收端物理层

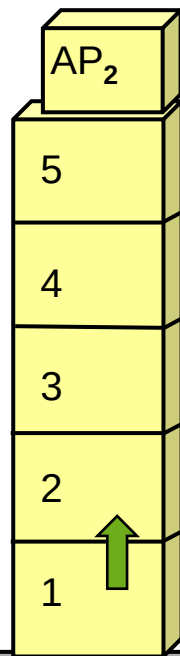


TCP/IP通信过程

主机 1



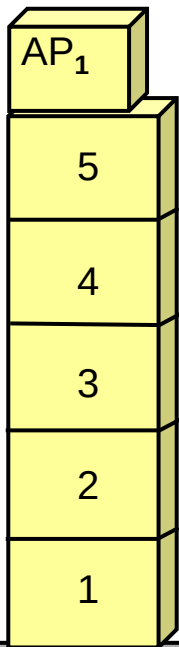
主机 2



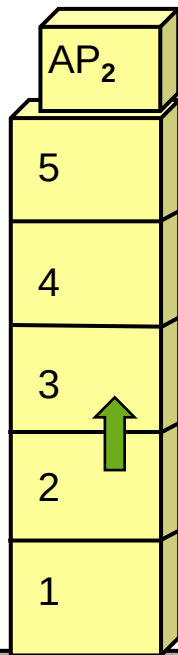
物理层接收到比特流，上交给数据链路层

TCP/IP通信过程

主机 1



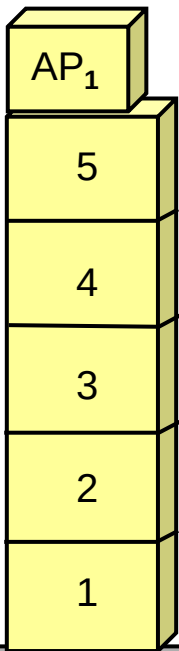
主机 2



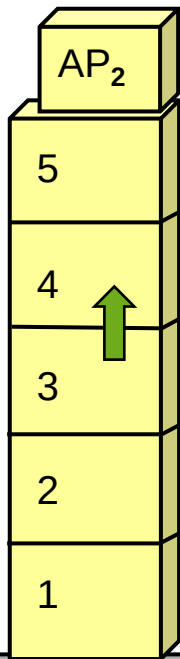
数据链路层剥去帧首部和帧尾部
取出数据部分，上交给网络层

TCP/IP通信过程

主机 1



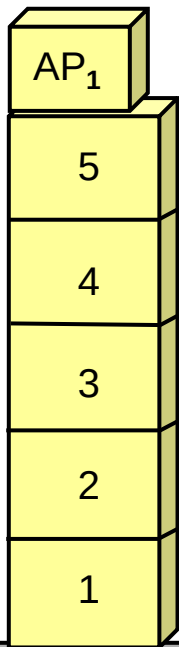
主机 2



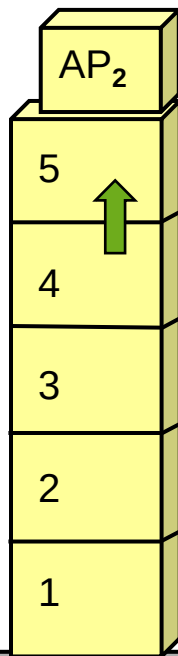
网络层剥去首部，取出数据部分
上交给运输层

TCP/IP通信过程

主机 1



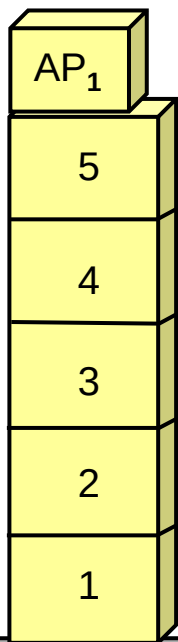
主机 2



运输层剥去首部，取出数据部分
上交给应用层

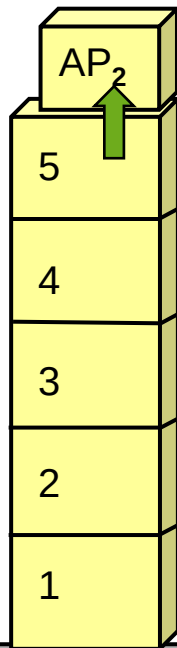
TCP/IP通信过程

主机 1

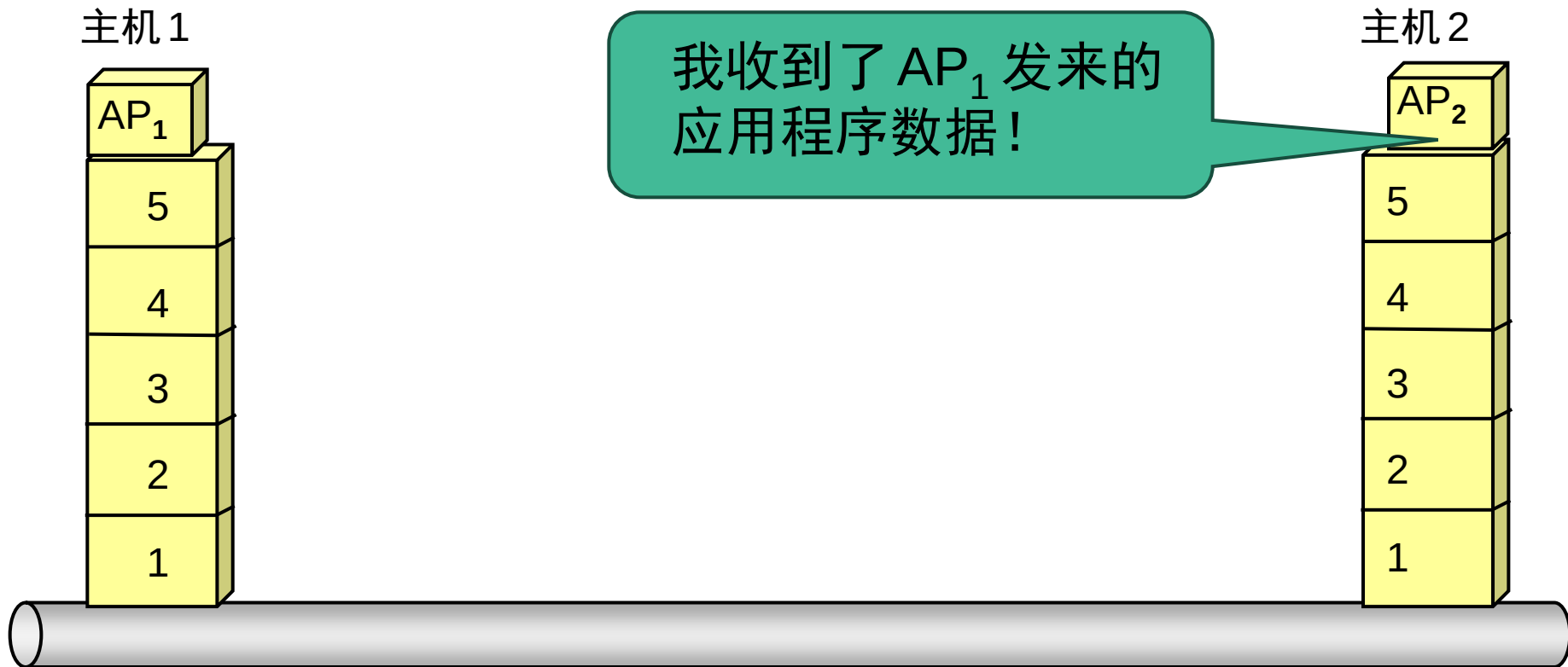


应用层剥去首部，取出应用程序数据
上交给应用进程

主机 2



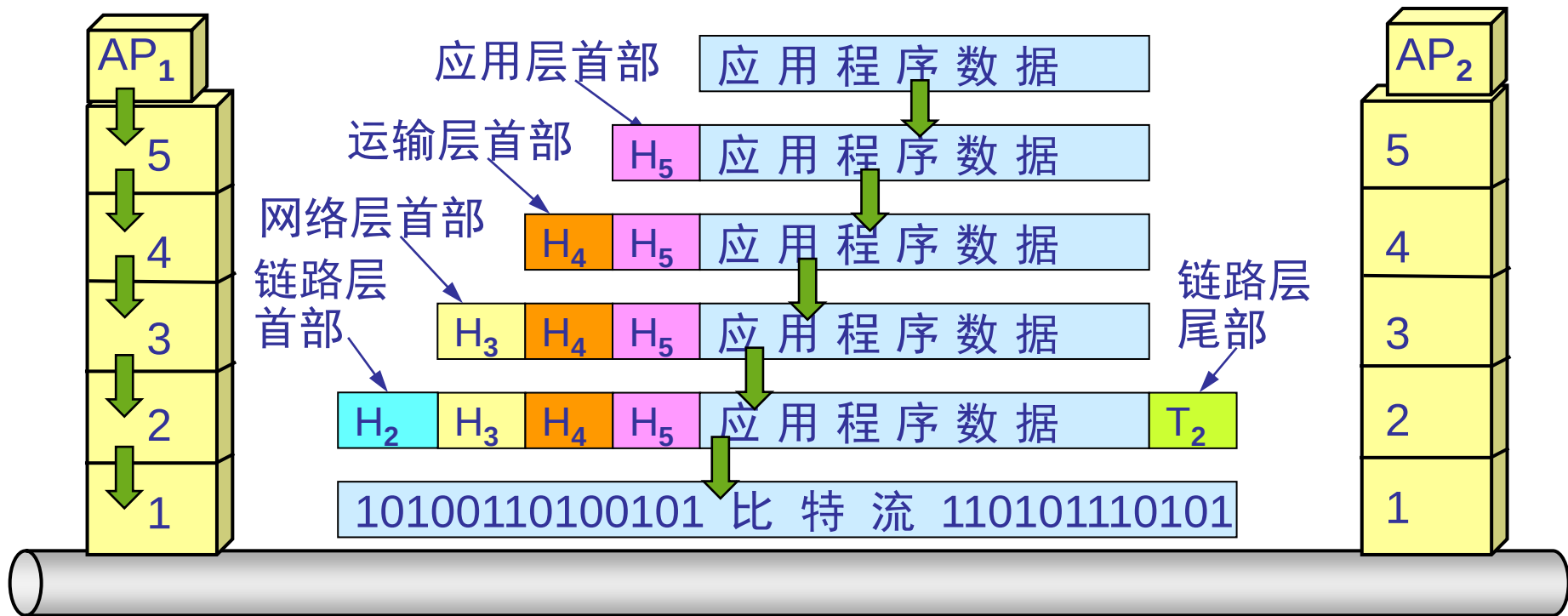
TCP/IP通信过程



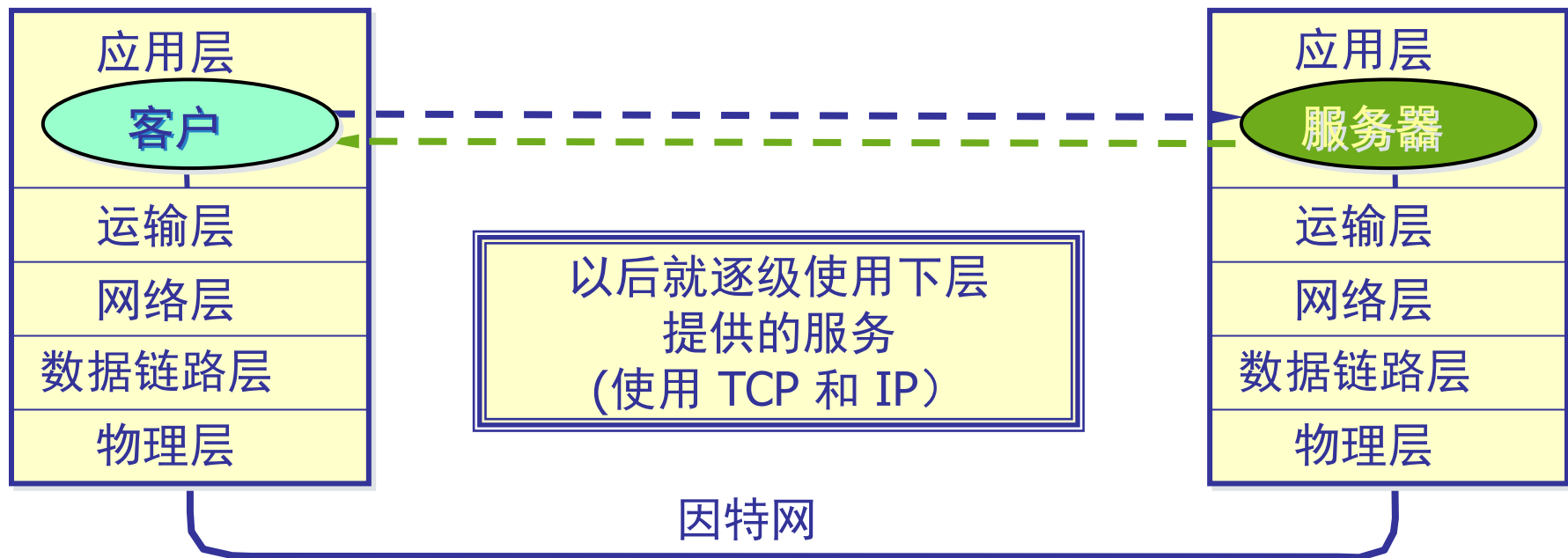
TCP/IP通信过程

主机 1

主机 2



TCP/IP通信过程——典型的C/S结构模型



网络编程接口的产生与发展

- 问题的提出

- 站在应用程序实现的角度，应用程序如何方便地使用协议栈软件进行通信呢？
- 如果能在应用程序与协议栈软件之间提供一个软件接口，就可以方便应用程序的开发。



网络编程接口的产生与发展

- 套接字编程接口

- 套接字应用程序编程接口是网络应用程序通过网络协议栈进行通信时所使用的接口，即应用程序与协议栈软件之间的接口，简称套接字编程接口(socket API)。
- 它定义了应用程序与协议栈软件进行交互时可以使用的一组操作，决定了应用程序使用协议栈的方式、应用程序所能实现的功能、以及开发具有这些功能的程序的难度。



socket 的历史

- 在80 年代早期，远景研究规划局（Advanced Research Projects Agency, ARPA）资助了佳利福尼亚大学伯克利分校的一个研究组，让他们将TCP/IP 软件移植到UNIX 操作系统中。
- 作为项目的一部分，设计者们创建了一个接口，应用进程使用这个接口可以方便的进行通信。
- 只要有可能就使用已有的系统调用，对那些不能方便的容入已有的函数集的情况，就再增加新的系统调用以支持TCP/IP 功能。

socket 的历史——**Berkeley Socket**

- 这个系统被称为Berkeley UNIX或BSD UNIX。(TCP/IP 协议栈首次出现在BSD 4.1 版本release 4.1 of Berkeley SoftwareDistribution)。
- 由许多计算机厂商，都采用了Berkeley UNIX，于是许多机器上都可以使用Socket了
- Socket 接口就被广泛使用，到现在已经成为事实上的标准。



Socket概述

Linux系统中的应用程序通过套接字编程接口来进行网络编程的.

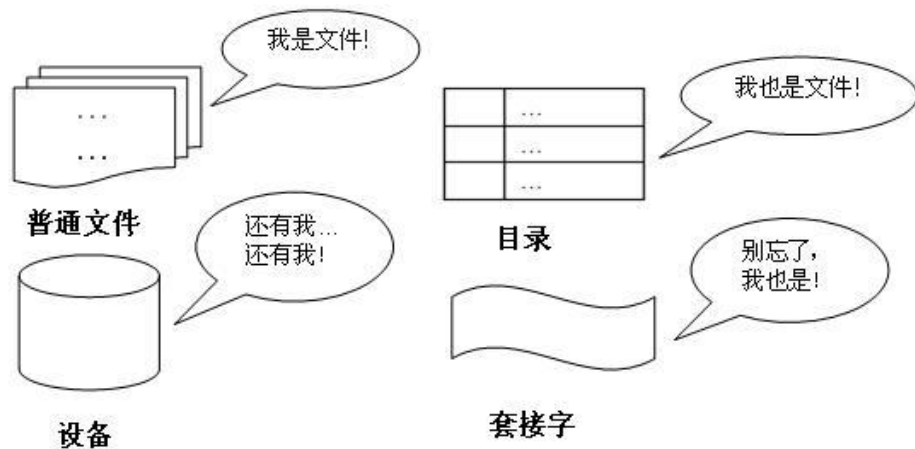
- 内核中的协议栈处理收到的各层协议数据，然后应用程序以**Socket文件的方式**接受内核传来的数据
- 应用程序通过**使用Socket API**，会返回Socket文件的描述符, 可以将这个描述符当做普通的文件的描述符来操作

可以通过文件读写操作实现网络之间的数据交流

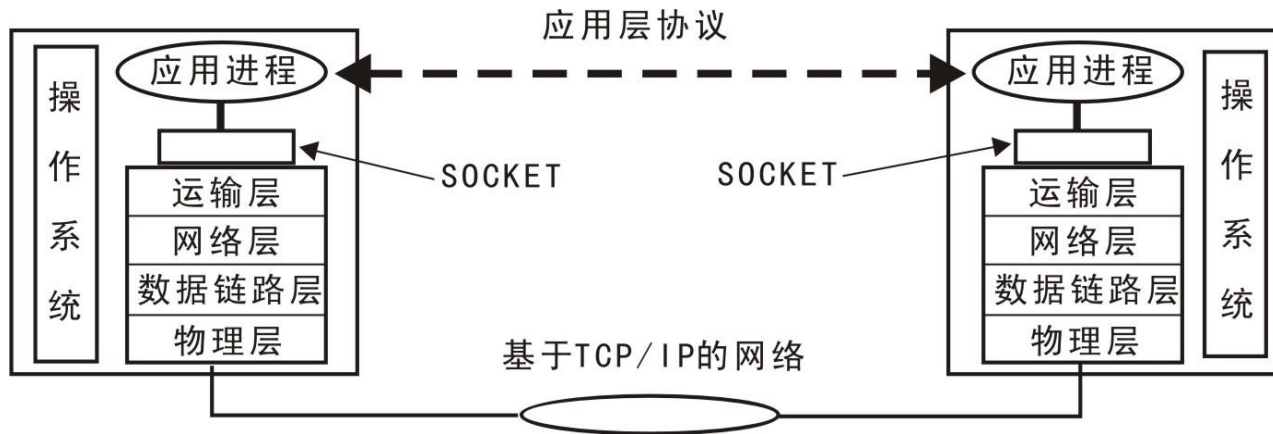


文件类型

标识	文件类型
-	普通文件
d	目录文件
c	字符设备文件
b	块设备文件
p	管道或FIFO
l	符号链接
s	套接字



Socket概述



- 应用进程通过Socket接口，使用传输层提供的服务，跨网络发送/接收消息到/从其他应用进程。
- Socket接口实现的程序模式是client/server模式



Socket概述

- Socket既是一种特殊的I/O，也是一种文件
- 一个完整的Socket 都有一个相关描述
 - 协议
 - 本地地址
 - 本地端口
 - 远程地址
 - 远程端口
- 每一个Socket 有一个本地的唯一Socket 号（文件描述符），由操作系统分配



- 流式套接字 (SOCK_STREAM)
 - 它提供基于TCP协议的双向、可靠、有序且不重复的无记录边界的数据流。
- 数据报套接字 (SOCK_DGRAM)
 - 它提供基于UDP协议的双向数据流，但不一定可靠、有序和不重复。
- 原始套接字
 - 它提供网络下层通信协议的直接访问。一般用于开发新的网络层协议，如新的IP协议等。

Socket地址结构

- 多数Socket API都需要是用一个指向地址结构的参数
- 对应于不同的协议簇，有不同的Socket地址结构
- 在Socket API中，专门定义了三种结构型的数据类型，用来存储协议相关的网络地址
 - struct sockaddr
 - struct sockaddr_in
 - struct in_addr



Socket地址结构

对应于不同的协议簇，有不同的地址结构。在头文件 `<linux/socket.h>` 中定义了以下结构来保持Socket API地址参数的一致性。

```
struct sockaddr{  
    unsigned short sa_family; /* 地址类型，格式为AF_XXX */  
    char sa_data[14]; /* 14字节的协议地址 */  
};
```

其中的sa_family为套接字的协议簇地址类型，TCP/IP的协议对于IPv4地址类型为AF_INET。sa_data中存储具体的协议地址，不同的协议簇有不同的地址格式。

Socket地址结构

但一般编程中并不直接针对socketaddr操作，而是使用另一个与sockaddr等价的数据结构sockaddr_in (在netinet/in.h中定义):

```
struct sockaddr_in{  
    unsigned short int sin_len; /* IPv4地址长度 */  
    short int sin_family; /* 地址类型 */  
    unsigned short int sin_port; /* 存储端口号 */  
    struct in_addr sin_addr; /* 存储IP地址 */  
    unsigned char sin_zero[8]; /* 空字节 */  
};
```

在 **sin_family** 指代协议族，在TCP套接字编程中只能是 **AF_INET**;

地址信息。

Socket地址结构

但一般编程中并不直接针对socketaddr操作，而是使用另一个与sockaddr等价的数据结构sockaddr_in (在netinet/in.h中定义):

```
struct sockaddr_in{  
    unsigned short int sin_len; /* IPv4地址长度 */  
    short int sin_family; /* 地址类型 */  
    unsigned short int sin_port; /* 存储端口号 */  
    struct in_addr sin_addr; /* 存储IP地址 */  
    unsigned char sin_zero[8]; /* 空字节 */  
};
```

sin_port存储端口号（使用网络字节顺序），数据类型是一个16位的无符号整数类型；

地址信息

Socket地址结构

但一般编程中并不直接针对socketaddr操作，而是使用另一个与sockaddr等价的数据结构sockaddr_in (在netinet/in.h中定义):

```
struct sockaddr_in{
    unsigned short int sin_len; /* IPv4地址长度 */
    short int sin_family; /* 地址类型 */
    unsigned short int sin_port; /* 存储端口号 */
    struct in_addr sin_addr; /* 存储IP地址 */
    unsigned char sin_zero[8]; /* 空字节 */
};
```

sin_addr存储IP地址，IP地址使用in_addr这个数据结构:

```
struct in_addr{ unsigned long s_addr; };
```

这个数据结构是由于历史原因保留下来，主要用作与以前的格式兼容。这里的s_addr按照网络字节顺序存储IP地址。

取地址信息

Socket地址结构

但一般编程中并不直接针对socketaddr操作，而是使用另一个与sockaddr等价的数据结构sockaddr_in (在netinet/in.h中定义):

```
struct sockaddr_in{  
    unsigned short int sin_len; /* IPv4地址长度 */  
    short int sin_family; /* 地址类型 */  
    unsigned short int sin_port; /* 存储端口号 */  
    struct in_addr sin_addr; /*存储IP地址 */  
    unsigned char sin_zero[8]; /* 空字节 */  
};
```

sin_zero是为了让sockaddr与sockaddr_in两个数据结构保持大小相同而保留的空字节。

地址信息

Socket地址结构

struct in_addr, 专门用来**存储IP地址**, 其定义如下:

- /* 因特网地址(a structure for historical reasons) */

```
struct sin_addr {  
  
    unsigned long s_addr;  
  
};
```

- 如果声明了 “ ina ” 作为一个struct sockaddr_in 的结构体变量, 那么
“ina.sin_addr.s_addr” 就是4个字节的IP 地址 (按网络字节顺序排放) 。



socket地址结构

这些数据结构的一般用法如下

- 首先，定义一个sockaddr_in的结构实例，并将它初始化为0。比如：

```
struct sockaddr_in mysocket;  
memset(& mysocket,0,sizeof(struct sockaddr_in));
```

- 然后，为这个结构赋值，比如：

```
mysocket.sin_family=AF_INET;  
mysocket.sin_port=htons(8080);  
mysocket.sin_addr.s_addr=inet_addr( "192.168.1.221" );
```

- 第三步：在函数调用中使用时，将这个结构强制转换为sockaddr类型。如：

```
accept(listenfd,(sockaddr*)& mysocket,&addrlen);
```



本机字节顺序与网络字节顺序

- 网络中存在多种类型的机器，这些不同类型的机器表示数据的字节顺序是不同的。
 - 以16进制数308A，在内存地址0x1000的存储为例

0x1001	30
0x1000	8A

小端字节序

0x1001	8A
0x1000	30

大端字节序

网络协议中的数据采用统一的网络字节顺序，因为只有采用统一的字节顺序，才能在不同类型的硬件设备之间正确的发送和接收数据。广域网规定的网络字节顺序采用大端字节顺序方式。

本机字节顺序与网络字节顺序

- Linux系统提供4个函数来进行字节顺序转换：

```
#include "netinet/in.h"
unsigned short int htons(unsigned short int hostshort);
unsigned long int htonl(unsigned long int hostlong);
unsigned short int ntohs(unsigned short int netshort);
unsigned long int ntohl(unsigned long int netlong);
```

其中。前两个函数将主机字节顺序转换成网络字节顺序；后两个函数将网络字节顺序转换成主机字节顺序。

在使用这些函数时，我们不关心主机或网络顺序的真实值到底是大端还是小端，只需要调用适当的函数来对给定值（函数的整型参数）进行主机字节顺序和网络字节顺序的转换，它们的返回值就是经过转换以后的结果。

IP地址转换函数

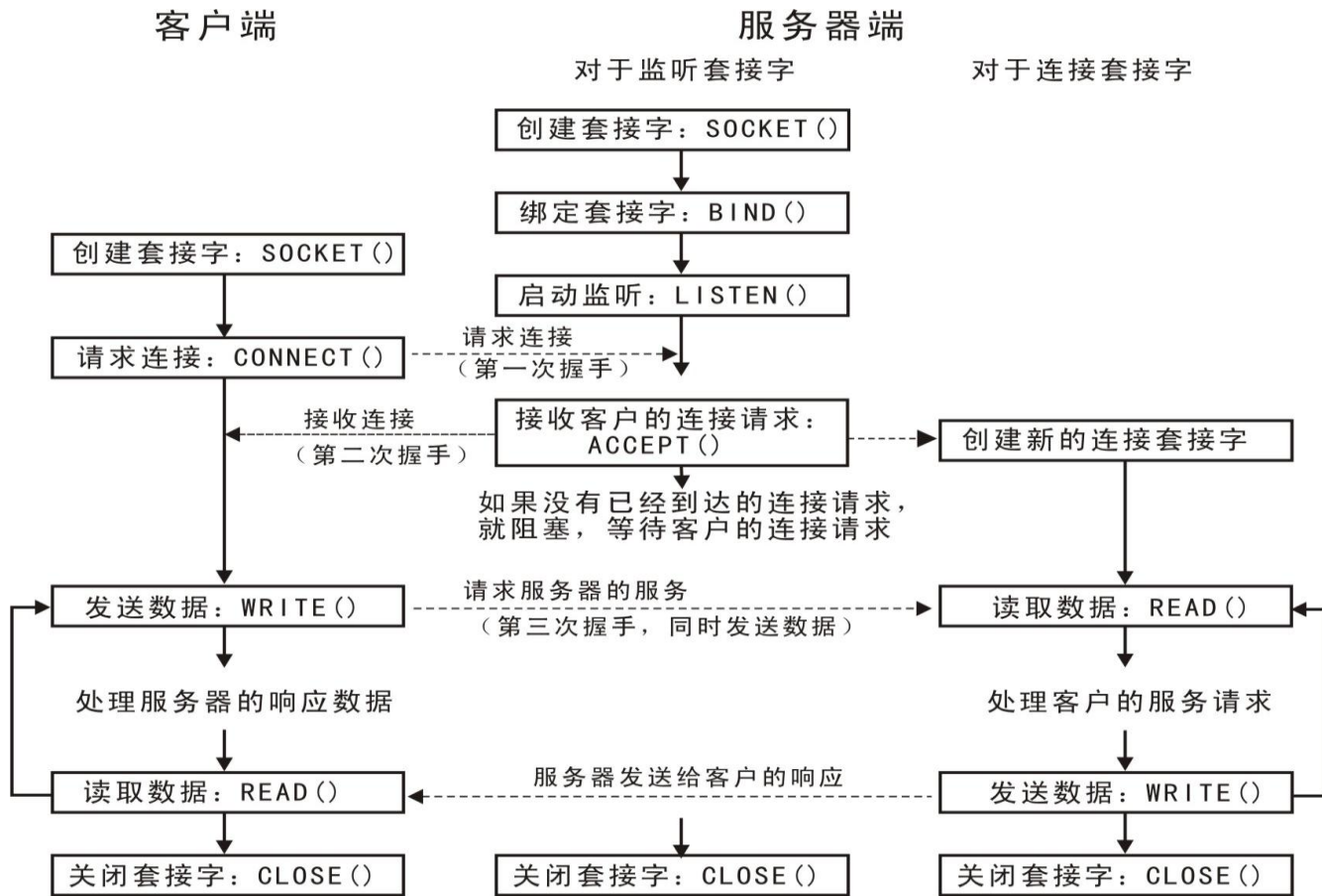
```
unsigned long inet_addr(const char *cp);
```

inet_addr将一个点分十进制IP地址字符串转换成32位数字表示的IP地址（网络字节顺序）。

```
char *inet_ntoa(struct in_addr in);
```

inet_ntoa将一个32位数字表示的IP地址转换成点分十进制IP地址字符串。

流式套接字 (TCP套接字) 工作流程



socket函数

- 为了执行网络输入输出，一个进程必须做的第一件事就是调用socket函数获得一个文件描述符（客户端和服务端程序均适用）

```
#include <sys/socket.h>
```

```
int socket(int family,int type,int protocol);
```

返回：非负描述字——成功 -1——失败

- ① 第一个参数指明了协议簇，目前支持5种协议簇，最常用的有AF_INET(IPv4协议)和AF_INET6(IPv6协议);
- ② 第二个参数指明套接口类型，有三种类型：SOCK_STREAM(字节流套接口)、SOCK_DGRAM(数据报套接口)和SOCK_RAW(原始套接口);
- ③ 如果套接口类型不是原始套接口，那么第三个参数就为0。如果套接口是SOCK_RAW时可设置不同的值（IPPROTO_IP、IPPROTO_ICMP等）

connect函数

- 当客户端程序调用socket获得了套接字对应的文件描述符后，可以调用connect为这个套接字指明远程端的地址
 - 如果是字节流套接字，connect就使用三次握手建立一个连接
 - 如果是数据报套接字，connect仅指明远程端地址，而不向它发送任何数据

```
#include <sys/socket.h>
int connect(int sockfd,const struct sockaddr * servaddr,socklen_t
addrlen);
返回：0——成功      -1——失败
```

- ① 第一个参数是socket函数返回的套接字对应的文件描述符；
- ② 第二和第三个参数分别是指向套接字地址结构的指针和该结构的大小。

bind函数

- 服务器端程序为套接字分配一个本地地址和协议端口
 - 对于IP协议，地址是32位IPv4地址（128位IPv6地址）与16位的TCP或UDP端口号的组合
 - 如指定端口为0，调用bind时内核将选择一个临时端口
 - 如果指定一个通配IP地址，则要等到建立连接后内核才选择一个本地IP地址。

```
#include <sys/socket.h>
int bind(int sockfd,const struct sockaddr * myaddr,socklen_t
addrlen);
返回：0——成功      -1——失败
```

- ① 第一个参数是socket函数返回的套接字对应的文件描述符
- ② 第二和第三个参数是指向特定协议的套接字地址结构的指针和该地址结构的长度。

listen函数

- 仅被服务器端程序调用，它的作用是将调用socket函数创建的主动套接字转换成被动套接字，并等待来自客户端的连接请求。

```
#include <sys/socket.h>
int listen(int sockfd,int backlog);
返回：0——成功      -1——失败
```

- ① 第一个参数是socket函数返回的套接字对应的文件描述符
- ② 第二个参数规定了内核为此套接口排队的最大连接个数
- ③ 由于listen函数第二个参数的原因，内核要维护两个队列已完成连接队列和未完成连接队列，当已连接队列为空时，进程将进入睡眠状态。

accept函数

仅被服务器端程序调用，从已完成连接队列头返回一个已完成连接，如果完成连接队列为空，则进程进入睡眠状态

```
#include <sys/socket.h>
```

```
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t * addrlen);
```

返回：非负描述字——成功 -1——失败

- ① 第一个参数是socket函数返回的套接字对应的文件描述符
- ② 第二个和第三个参数是指向连接方的套接字地址结构和该地址结构的长度（如果对客户段的信息不感兴趣，可以将第二和第三个参数置为空）
- ③ 该函数返回的是一个全新的套接字对应的文件描述符，可以用不同的文件描述符来区别多个客户端与服务器的不同连接

头文件unistd.h

```
ssize_t read( int filedes, void *buf, size_t nbytes);
```

- read函数从套接字中读数据。如成功，则返回实际读到的字节数，如无数据可读，则返回0；有多种情况可使实际读到的字节数少于要求读的字节数：
 - 当从网络读时，网络中的缓冲机构可能造成返回值小于所要求读的字节数；
- 进程调用read函数从套接字上读取数据，当套接字的接收缓冲区中还没有数据可读，函数read将发生阻塞，等待套接字的接收缓冲区中有数据可读；经过一段时间后，缓冲区内接收到数据，于是内核便去唤醒该进程，通过read访问这些数据。
- 如果在进程阻塞过程中，对方发生故障，那这个进程将永远阻塞下去。

头文件unistd.h

ssize_t write(int filedes, const void *buf, size_t nbytes);

- 该函数返回实际写的字节数，通常与参数nbytes的值相同，否则表示出错。
- 如果出错，则返回 - 1
- 在写操作时发生阻塞的情况要比读操作少。主要发生在要写入的缓冲区的大小小于要写入的数据量的情况下。这时，写操作不进行任何拷贝工作，将发生阻塞
- 一旦发送缓冲区内有足够的空间，内核将唤醒进程，将数据从用户缓冲区中拷贝到相应的发送数据缓冲区。

数据传输

- `int send(int sockfd, const void * data, int data_len, unsigned int flags)`

✚ 功能:

- ✱ 在TCP连接上发送数据（服务器和客户端均适用）,返回成功传送数据的长度，出错时返回 - 1
- ✱ send会将外发数据复制到OS内核中，也可以使用send发送面向连接的UDP报文。

✚ 参数说明:

- ◆ sockfd:套接字对应的文件描述符（对于客户端是socket函数的返回值，对于服务器是connet函数的返回值）
- ◆ data:指向要发送数据的指针
- ◆ data_len:数据长度
- ◆ flags:一般为0，则和write一样的操作

- `int recv(int sockfd, void *buf, int buf_len, unsigned int flags);`

- ✦ 功能:

- ✦ 从TCP接收数据,返回实际接收的数据长度, 出错时返回 - 1。
- ✦ 服务器使用其接收客户请求, 客户使用它接受服务器的应答。如果没有数据, 将阻塞, 如果收到的数据大于缓存的大小, 多余的数据将丢弃。也可以使用recv接收面向连接的UDP的报文

- ✦ 参数说明:

- ◆ Sockfd:套接字对应的文件描述符 (对于客户端是socket函数的返回值, 对于服务器是connet函数的返回值)
- ◆ Buf:指向缓冲区内存的指针
- ◆ Buf_len:缓冲区大小, 以字节为单位
- ◆ flags:一般为0, 则和read一样的操作

■ send, recv函数flag标志

MSG_DONTROUTE	不查找路由表
MSG_OOB	接受或者发送带外数据
MSG_PEEK	查看数据,并不从系统缓冲区移走数据
MSG_WAITALL	等待所有数据

- MSG_WAITALL是recv函数的使用标志, 使用这个标志的时候recv会一直阻塞, 直到指定的条件满足, 或者是发生了错误。
 - 1) 当读到了指定的字节时, 函数正常返回.返回值等于len。
 - 2) 当读到了文件的结尾时, 函数正常返回, 返回值小于len。
 - 3) 当操作发生错误时, 返回-1, 且设置错误为相应的错误号(errno)

- close()函数

- 用来关闭一个套接字描述符。当完成了套接字的建立、绑定、连接和通信读写之后，都需要最后关闭建立的套接字

```
#include <unistd.h>
```

```
int close(int sockfd);
```

- sockfd是需要关闭的套接字所对应的文件描述符。若执行成功则返回0，否则返回-1。
- 清除套接字描述符的具体操作：将这个套接字描述符标记为CLOSED状态，然后立即返回进程。TCP协议将继续使用这个套接字，将尚未发送的数据传递到对方，然后发送FIN数据段，关闭和释放这个套接字。这是套接字才被完全删除。

- shutdown()函数

- 可以部分关闭socket连接，单方面的中断连接，即禁止某个方向的信息传递。

```
#include <unistd.h>
```

```
int shutdown(int sockfd,int how);
```

- sockfd代表需要关闭的套接字描述符；how的值：
 - 0：禁止接收信息
 - 1：禁止发送信息
 - 2：接收和发送都被禁止，与close()函数效果相同。

- close()和shutdown()的区别：
 - close()函数调用时将**访问计数减1**，只有当访问计数减少到0以后，才关闭套接字；而shutdown()函数可直接发送FIN数据段以终止网络连接，而不管访问计数的多少。
 - **shutdown()实际上只关闭了TCP连接的一半**。因为TCP连接是双向的，即包括读和写两个方向。shutdown()函数可以给服务器发送一个FIN，告诉它已经完成了数据发送，但仍为读而开放套接字描述符。close()函数则是同时关闭了读写两个方向。

训练1

- 训练内容-开发基于TCP的文本聊天程序

文本聊天程序的双方分别是服务器和客户端（服务器和客户端运行在两个Ubuntu虚拟机中）

1.服务器

服务器程序等待客户的连接，客户连接成功后可以与客户进行文字通信（显示通信内容）。服务器如果收到客户方的exit文本，则退出服务器程序

2.客户端

建立一个用TCP连接的客户，以命令行的方式启动，命令行要求有服务器地址，服务器端口号。连接建立成功后可以输入文本与服务器进行通信（显示通信内容）。客户端通过输入exit文本结束与服务器的通信并退出程序



Webserver V0.6版

网络服务模块



mini_webserver

工程基础模块 Day1 && Day4

构建子模块 D1
Makefile
编译、清理、调试版本

配置子模块 D1
Config.c
端口、根目录、线程数

日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查

CSV存储子模块 D2
csv_store.c
用户数据加载和保存

索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

TCP监听子模块 D6
server.c
socket, bind, listen

连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

进程模型子模块 D4
process.c
父子进程协同

线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护

线程池子模块 D8
thread_pool.c
任务队列、工作线程

事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

请求解析子模块 D10
http_request.c
方法、URL、Header、Body

静态资源子模块 D11
static_file.c
HTML、CSS、图片读取

GET/POST处理子模块 D12
handler.c
表单提交、参数读取

路由配置子模块 D13
router.c
URL匹配、处理函数绑定

认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查

响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

每日增加功能的相关测试

如: test_day01.sh

如: bench.c 并发压力测试/响应时间等

Web Server V0.6开始,

- 模拟的请求请求不再来自requests/hello.req, 而是来自本机模拟的TCP连接。
- conf/server.conf 里的内容真正用于程序中的socket绑定

```
server_addr.sin_port = htons(config->port);  
server_addr.sin_addr.s_addr = inet_addr(config->host);
```



使用curl命令模拟http请求:

```
curl http://127.0.0.1:8080/hello
```

CURL的官方定义为:curl is a command line tool for transferring data with URL syntax,即使用URL语法规则来传输数据的命令行工具。



浏览器或 curl 发来的内容类似:

```
GET /hello HTTP/1.1  
Host: 127.0.0.1:8080  
User-Agent: curl/...
```

本次先只处理第一行

新增文件包括:

- `include/tcp_server.h`
- `src/tcp_server.c`
- `tests/test_day06.sh`

修改文件包括

- `rc/main.c`
- `src/request_handler.c`
- `include/request_handler.h`
- `Makefile`
- `README.md`
- `docs/debug-log.txt` (可选)



V0.6版 mini_webserver-功能检查

- curl /hello 能收到 HTTP/1.1 200 OK
- curl /users/zhangsan 能收到用户信息
- curl /not-exist 能收到 HTTP/1.1 404 Not Found
- 服务器处理一个连接后正常退出
- 能理解socket、bind、listen、accept 的作用

